

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра Информационные системы

ОТЧЕТ
по лабораторной работе №1
по дисциплине «Информатика и информационные технологии»
Тема: Простой интерпретатор арифметических действий

Студент гр. 6373

Толоконцев К.И.

Преподаватель

Дюков Н.В.

Санкт-Петербург

2026

Цель работы.

Разработать на языке Python интерпретатор арифметических выражений, способный принимать строку с математическим выражением и выдавать итоговое числовое значение.

Основные теоретические положения.

- Интерпретатор выражений: программа, которая разбирает текст математического выражения, строит его синтаксическое представление и вычисляет результат.
- Лексер: Часть компилятора или интерпретатора, которая отвечает за разбиение входной строки на отдельные элементы — токены или лексемы.
- Обратная польская запись — это способ записи математических формул, где знак операции (плюс, минус и другие) стоит после чисел, с которыми она проводится.

Разработка программы

- `is_digit`: Проверяет является ли строка числом
- `split_raw`: Регулярное выражение которое разбивает выражение на токены
- `remove_m`: Унарный минус относящийся к числу записывает в его токен, унарный минус относящийся к оператору превращает в унарный оператор.
- `toemreverse_polish`: преобразует токены нормальной формы в токены обратной польской. Операторы записываются в стек. если у оператора меньше приоритет чем у оператора в стеке то оператор из стека переходит в `output`. числа сразу записываются в `output`. Отдельно обрабатываются `max` и `min` так как у них может быть любое количество аргументов которое записывается в их имя для удобства.
- `calculate`: Преобразует обратную польскую в число. Числа добавляются в стек, а операторы достают нужное им количество аргументов. Аналогично сделано для прямой польской записи.

Результаты работы программы

Ввод	Вывод
1 + 1	2
+ 1 1	2
1 1 +	2
-sqrt(abs(1 - sin(2)))	-0.30116867893

Выводы.

Был разработан интерпретатор математических выражений который успешно может вернуть значение математического выражения.

ПРИЛОЖЕНИЕ

```
from math import *  
import re
```

```
functions = [  
    "pow",  
    "round",  
    "sqrt",  
    "abs",  
    "sin",  
    "cos",  
    "tan",  
    "log",  
    "exp",  
    "floor",  
    "ceil",  
    "fact",  
    "log10",  
]
```

```
oper = {  
    "and": 0,  
    "|": 1,  
    "^": 2,  
    "&": 3,  
    "=": 4,  
    "≠": 4,  
    "≥": 5,  
    "≤": 5,  
    "<": 5,  
    ">": 5,  
    "<<": 6,  
    ">>": 6,  
    "+": 7,  
    "-": 7,  
    "*": 8,  
    "/": 8,  
    "m": 9,  
}
```

```
def is_digit(n):
```

```

return n.replace(".", "").replace("-", "").isdigit()

# Разбиваем на части
def split_raw(raw_str):
    regular = re.compile(
        r"""
        \d+(?:\.\d+)?|
        [a-z]\w*|
        ≥|≤|=|≠|<<|>>|
        [\+\-\*/<>^\(\)]
        """,
        re.VERBOSE,
    )
    return re.findall(regular, raw_str)

# Убираем унарный минус
def remove_m(badtokens):
    tokens = []
    i = 0
    while i < len(badtokens):
        if badtokens[i] == "-" and (
            i == 0
            or badtokens[i - 1] == "("
            or badtokens[i - 1] == ","
            or badtokens[i - 1] in oper
        ):
            if is_digit(badtokens[i + 1]):
                tokens.append("-" + badtokens[i + 1])
                i += 1
            else:
                tokens.append("m")
        else:
            tokens.append(badtokens[i])
        i += 1
    return tokens

# В обратную польскую
def to_reverse_polish(tokens):
    output = []

```

```

stack = []
i = 0

while i < len(tokens):
    if is_digit(tokens[i]):
        output.append(tokens[i])
    elif tokens[i] == "(":
        stack.append(tokens[i])
    elif tokens[i] == ")":
        while stack and stack[-1] != "(":
            output.append(stack.pop())
        stack.pop()
    elif tokens[i] in oper:
        while stack and stack[-1] in oper and oper[stack[-1]] >=
oper[tokens[i]]:
            output.append(stack.pop())
        stack.append(tokens[i])
    elif tokens[i] in functions:
        stack.extend(["(", tokens[i]])
        i += 1
    elif tokens[i] == "max" or tokens[i] == "min":
        countc = 0
        counts = 1
        j = i + 1
        while counts:
            j += 1
            if tokens[j] == "(":
                counts += 1
            elif tokens[j] == ")":
                counts -= 1
            elif counts == 1:
                countc += 1
        stack.extend(["(", tokens[i] + str(countc)])
        i += 1

    i += 1
while stack:
    output.append(stack.pop())
return output

```

```

loper1 = {

```

```

    "sqrt": sqrt,
    "abs": fabs,
    "sin": sin,
    "cos": cos,
    "tan": tan,
    "log": log,
    "exp": exp,
    "floor": floor,
    "ceil": ceil,
    "fact": factorial,
    "log10": log10,
    "m": lambda a: -a,
}

loper2 = {
    "|": lambda a, b: int(a) | int(b),
    "^": lambda a, b: int(a) ^ int(b),
    "&": lambda a, b: int(a) & int(b),
    "==": lambda a, b: int(a) == int(b),
    "!=": lambda a, b: int(a) != int(b),
    ">=": lambda a, b: float(a) >= float(b),
    "<=": lambda a, b: float(a) <= float(b),
    "<": lambda a, b: float(a) < float(b),
    ">": lambda a, b: float(a) > float(b),
    "<<": lambda a, b: int(a) << int(b),
    ">>": lambda a, b: int(a) >> int(b),
    "+": lambda a, b: float(a) + float(b),
    "-": lambda a, b: float(a) - float(b),
    "*": lambda a, b: float(a) * float(b),
    "/": lambda a, b: float(a) / float(b),
    "and": lambda a, b: int(a) and int(b),
    "pow": lambda a, b: int(a) ** int(b),
    "round": lambda a, b: round(float(a), int(b)),
}

```

Считаем обратную польскую

```

def calculate(tokens):
    stack = []

    for e in tokens:
        if is_digit(e):

```

```

        stack.append(e)
    elif e in loper1:
        stack.append(loser1[e](float(stack.pop()))))
    elif e in loper2:
        a = stack.pop()
        b = stack.pop()
        stack.append(loser2[e](b, a))
    elif e[0:3] == "max":
        n = int(e[3:])
        maxel = max([float(stack.pop()) for i in range(n)])
        stack.append(maxel)
    elif e[0:3] == "min":
        n = int(e[3:])
        maxel = min([float(stack.pop()) for i in range(n)])
        stack.append(maxel)

return stack[0]

```

Считаем прямую польскую

```

def calculate2(tokens):
    stack = []

    for e in tokens[::-1]:
        if is_digit(e):
            stack.append(e)
        elif e in loper1:
            stack.append(loser1[e](float(stack.pop()))))
        elif e in loper2:
            a = stack.pop()
            b = stack.pop()
            stack.append(loser2[e](a, b))
        elif e[0:3] == "max":
            n = int(e[3:])
            maxel = max([float(stack.pop()) for i in range(n)])
            stack.append(maxel)
        elif e[0:3] == "min":
            n = int(e[3:])
            maxel = min([float(stack.pop()) for i in range(n)])
            stack.append(maxel)

    return stack[0]

```

```

raw_str = str(input())
tokens = split_raw(raw_str)
# print(tokens)
if is_digit(tokens[-1]) or tokens[-1] == ")":
    new_tokens = remove_m(tokens)
    print(tokens)
    if (
        is_digit(new_tokens[0])
        or new_tokens[0] == "m"
        or new_tokens[0] in functions
        or new_tokens[0] == "("
    ):
        print("Результат:", calculate(to_reverse_polish(new_tokens)))
    else:
        print("Результат:", calculate2(tokens)) # прямая
else:
    print("Результат:", calculate(tokens)) # обратная

```